Law Firm Case Management System

CSCI 377 Project Description ·

Team Members: Shafin Awal, Adam Mohamed

Project Description

We will build a case management system for a law firm in Python. It will handle the firm's core operations: opening new cases, managing client records, looking up billing, updating case status, and closing or deleting cases. The main goal is to measure how the data structures and algorithms behind these operations actually perform on a dataset of one million case records, and to compare those measured running times with their theoretical time complexities.

Dataset

We will generate one million synthetic case records using Python's `random` module and the Faker library, with a fixed seed so that every run uses the same data. To keep generation fast, Faker will produce a pool of about 5,000 client names once, and records will draw from that pool. Each record contains a Case ID, client name, filing date, case type, status (Open, Pending, Closed) and amount billed.

To keep memory use manageable at this scale, each record will be stored as a `dataclass` with `slots=True` rather than as a dictionary. The generated dataset will be saved to disk so that it doesn't have to be rebuilt for every experiment.

Data Structures

- **Hash table (Python `dict`):** keyed by Case ID. It supports average-case O(1) insertion, lookup and deletion, which makes it the backbone for status updates and direct case retrieval.
- **Binary search tree (BST):** keyed by filing date. It keeps cases in date order and supports range queries such as "all cases filed in March 2026." Insert, search and delete take O(log n) on average but O(n) in the worst case.
- **AVL tree:** a self-balancing BST built on the same key. Rotations keep the tree's height at O(log n), which guarantees O(log n) for every operation. We will run both trees on the same random input and on the same date-sorted input, to show the plain BST collapsing toward a linked list while the AVL tree stays balanced.
- **Array (Python list):** holds the raw records for benchmarking the sorting and search algorithms.

Algorithms

1. **Merge Sort:** O(n log n) in every case. It sorts cases by filing date or alphabetically by client name, and it is stable, so cases with the same date keep their relative order.

2. **Quick Sort:** O(n log n) on average and O(n^2) in the worst case. We will use a random pivot and three-way partitioning, because many cases share a filing date and a naive Quick Sort slows down sharply on duplicate or already-sorted keys.
3. **Binary Search:** O(log n) on a sorted array. It will be compared against linear search (O(n)) and hash table lookup (O(1) on average).
4. **BST and AVL operations:**
    - Insertion.
    - Search.
    - Deletion, covering all three cases: a node with no children, with one child, and with two children (replaced by its in-order successor).
    - In-order traversal, to list cases in date order.
    - Range queries, used for billing reports over a period.

All tree operations will be written as loops rather than recursion, so that a degenerate tree cannot exceed Python's recursion limit.

How System Operations Map to Structures

Operation	Structure used	Expected cost
Open a new case	Hash table and AVL tree	O(1) and O(log n)
Look up a case by ID	Hash table	O(1)
Update case status	Hash table lookup, then edit in place	O(1)
Billing total for a date range	AVL tree range query	O(log n + k), where k is the number of cases in the range
Close or delete a case	Hash table and AVL tree	O(1) and O(log n)
List cases by client name	Merge Sort on the array	O(n log n)

Experimental Setup

The project will be implemented in Python 3.11 or later. Faker will be used for data generation, matplotlib for visualization and pytest for correctness testing. All sorting, searching and tree algorithms will be implemented from scratch; no library sorting, tree or search functions will be used in the graded components.

Timing will use `time.perf_counter()`. Every test will run five times, and we will report the median. Generating the data will not be included in any timing.

- **Input sizes:** 10,000, 100,000, 500,000 and 1,000,000 records.
- **Input orders:** random, already sorted and reverse sorted.
- **Searches:** both cases that exist and cases that do not, so that worst-case behavior is captured.

- **Reference point:** Python's built-in `sorted()` (Timsort), for comparison with our own sorts.

Correctness will be checked with pytest. For example, the tests will check that our sorts produce the same output as `sorted()`, and that the AVL tree stays balanced after a million insertions. The results will be plotted with matplotlib against the predicted complexity curves, so that any gaps between theory and measurement can be identified and explained.

Timeline

Date	Milestone
10/18	Data generator and record structure complete
10/24	Hash table operations and BST complete; **Progress Meeting 1** (one-third of the project)
11/5	AVL tree, Merge Sort, Quick Sort and search algorithms complete
11/12	Benchmarking setup complete; **Progress Meeting 2** (two-thirds of the project)
11/24	All experiments run, graphs produced, command-line demo finished
12/1–12/3	Presentation and live demo
12/1–12/13	Peer review of another group's project
12/14	Final report, code and README submitted

Division of Work

- Shafin Awal: _____
- Adam Mohamed: _____

AI Disclosure

Generative AI (Claude) was used to brainstorm the experimental design and to review this draft for gaps. All design decisions, code and analysis are our own work.